Universidade Católica de Brasília

Pró-Reitoria Acadêmica
Curso de Engenharia de Software

RELATÓRIO DA ETAPA 2 — OPERAÇÃO, INTERFACE E DESEMPENHO

Discente: Igor Santiago Pereira da Silva,
João Arantes Rodrigues,
Jander Matheus Correia de Lima Pinto
Docente: Samuel Novais Moura Júnior

Brasília - DF
2026

RELATÓRIO DA ETAPA 2 — OPERAÇÃO, INTERFACE E DESEMPENHO

1 Introdução

Na Etapa 1 o banco de dados da distribuidora de bebidas foi modelado, implementado e carregado com dados. Nesta segunda etapa ele sai do papel e entra em operação: foi construída uma aplicação que o utiliza de verdade, as operações mais sensíveis foram colocadas dentro de transações, o comportamento do banco sob acesso simultâneo foi observado e o controle de acesso começou a ser definido.

Vale deixar claro desde o início o papel da aplicação neste trabalho. Ela não é o objeto da avaliação, e não foi feita para impressionar pela aparência. Ela é o instrumento que permite demonstrar que o banco funciona — que as operações são transacionais, que as consultas são parametrizadas, que os erros de restrição são tratados e que o acesso respeita perfis. Por isso a interface é simples e direta, e todas as decisões técnicas foram tomadas no sentido de deixar o banco visível, e não de escondê-lo.

As treze tabelas da Etapa 1 continuam as mesmas, com as mesmas restrições nomeadas e os mesmos índices. Houve duas alterações, ambas decorrentes da devolutiva e descritas na seção 9: a coluna *preco_comprar* passou a chamar-se *preco_compra*, e cada restrição recebeu um comentário ligando-a à regra de negócio que atende. Nenhuma tabela, chave ou tipo de dado mudou.

O que esta etapa acrescentou ao banco foram três objetos novos, todos reunidos no arquivo *04_etapa2.sql*.

O primeiro é a tabela *usuario*, que guarda quem opera o sistema, com a senha protegida por hash e sal. Ela não se confunde com os perfis de acesso do próprio MySQL, criados com GRANT e REVOKE: aqui está quem usa a aplicação, lá está quem se conecta ao banco. Os outros dois são visões. A *vw_faturamento_categoria* alimenta a tela de relatório e concentra a regra de apuração do faturamento, inclusive a exclusão dos pedidos cancelados exigida pela RN19. A *vw_posicao_estoque* reúne o saldo de cada produto com o seu nome e a sua categoria, e é dela que a tela de venda lê.

2 A aplicação

A aplicação é um programa de computador em Java 21, com interface em Swing, que se conecta ao MySQL por JDBC puro. A decisão de não usar um mapeador objeto-relacional foi deliberada. Um mapeador geraria o SQL automaticamente e esconderia exatamente aquilo que esta disciplina avalia. Sem ele, o esquema continua sendo o arquivo escrito pela equipe e todo o SQL da aplicação está visível nas classes de acesso a dados, podendo ser lido, explicado e ter o seu plano de execução analisado.

São cerca de três mil linhas distribuídas em trinta e seis classes, organizadas em quatro pacotes: um para a conexão e a tradução de erros, um para a guarda de senha, um para todo o SQL e um para as telas.

2.1 Os cadastros

O enunciado pede cadastro completo de pelo menos quatro entidades. A aplicação atende cinco: produtos, categorias, fornecedores, clientes e funcionários. Cada uma permite criar, consultar com filtro, alterar e remover.

Três desses cadastros gravam em mais de uma tabela, e por isso funcionam dentro de uma transação. Cliente e funcionário são especializações de pessoa, conforme definido na RN17, de modo que salvar qualquer um dos dois significa escrever em duas tabelas. O produto é um caso que merece explicação, porque revelou um erro durante os testes.

Inicialmente o cadastro gravava apenas na tabela de produto. O resultado era um registro pela metade: como a tela de venda lê da visão que junta produto e estoque, um produto criado pela tela existia no cadastro mas era invisível para a venda, sem qualquer mensagem de erro. O cadastro passou então a criar, na mesma transação, a posição de estoque do produto, com saldo zero. Além de corrigir o problema, essa é a representação correta do domínio: o produto só passa a ter saldo quando entra mercadoria, e até lá a RN11 impede que ele seja vendido.

A exclusão de produto seguiu o mesmo raciocínio e precisou respeitar a ordem das chaves estrangeiras. A chave que liga estoque a produto impede apagar o produto enquanto houver estoque, então a posição de estoque sai primeiro. Já a chave que liga movimentação a estoque apaga em cascata, de modo que remover a posição leva

junto o histórico de movimentações. Se o produto já tiver sido vendido, a exclusão falha no segundo comando — e sem transação o histórico teria sido apagado por uma operação que no fim não se completou. Com a transação, o desfazimento devolve tudo ao estado anterior.

Todos os cadastros usam a mesma tela, que monta a tabela e o formulário a partir da descrição que cada classe de acesso fornece. Assim o SQL fica concentrado em um lugar só e a tela cuida apenas da interação com o usuário.

2.2 As consultas

São duas telas de consulta, ambas com junção entre tabelas e agregação de valores, e ambas recebendo filtros informados pelo usuário.

A primeira mostra quanto cada categoria faturou em um período. O usuário informa a data inicial, a data final e, se quiser, parte do nome da categoria. A consulta junta as tabelas de categoria, produto, item de pedido e pedido, soma os valores por categoria e deixa de fora os pedidos cancelados, como manda a RN19.

A segunda responde quais produtos precisam ser recomprados. O usuário informa uma margem de segurança em dias, e a consulta compara a cobertura de estoque de cada produto com o prazo de entrega do seu fornecedor mais lento, somado a essa margem. É a mesma pergunta de negócio não trivial defendida na Etapa 1, agora com o parâmetro ajustável pela tela.

2.3 O relatório

A tela de relatório faz uma única consulta na visão de faturamento por categoria e apresenta o total geral. Nenhuma regra de apuração está escrita no programa: quem decide o que entra no faturamento é o banco, através da visão. Se a regra mudar, muda a visão, e a tela continua igual.

2.4 As mensagens de erro

Quando o banco recusa uma operação, o usuário precisa entender o motivo. A exceção que o driver devolve é técnica e não serve para isso.

A solução se apoia em uma decisão tomada lá na Etapa 1: todas as restrições do banco receberam nome. Como o MySQL devolve esse nome dentro da mensagem de erro, a aplicação consegue identificar qual regra foi violada e responder a ela especificamente. Nomear as restrições, portanto, não foi apenas capricho de padronização — é o que torna possível dar uma resposta útil.

Quem tenta cadastrar um CPF repetido lê que já existe uma pessoa com esse CPF. Quem tenta vender mais do que existe lê quanto há disponível. Quem tenta excluir um produto já vendido, ou uma categoria que ainda tem produtos, lê exatamente isso. São cerca de trinta restrições mapeadas. Quando nenhuma é reconhecida pelo nome, a tradução recorre aos códigos gerais do MySQL, que ainda dizem bem mais do que a exceção original.

3 O acesso ao banco de dados

3.1 Consultas parametrizadas

Nenhuma consulta da aplicação é montada juntando pedaços de texto. Todo valor informado pelo usuário entra como parâmetro, inclusive o conteúdo das buscas por parte do nome, em que os curingas são acrescentados ao valor e não ao comando. O único dado que a aplicação monta em código é o identificador vindo de uma caixa de seleção, que nunca foi digitado por ninguém — e mesmo ele entra como parâmetro numérico.

3.2 Credenciais fora do código

O usuário e a senha do banco ficam em um arquivo de configuração, lido quando a aplicação começa a rodar. Esse arquivo está na lista de exclusão do controle de versão e nunca foi para o repositório. O que é versionado é apenas um arquivo de exemplo, com a senha substituída por um lembrete.

Na primeira vez que a aplicação roda em uma máquina nova, ela abre uma janela pedindo servidor, porta, banco, usuário e senha. Antes de gravar qualquer coisa, ela testa a conexão: se a senha estiver errada, o erro

aparece ali, com a causa explicada, e não no meio de uma operação. Isso evita a alternativa comum de deixar uma senha de exemplo escrita no código para quem for executar trocar depois — que, na prática, acaba indo para o repositório.

3.3 Teste de injeção de SQL

Para verificar se a aplicação resiste a uma tentativa de injeção, foi feito o seguinte teste. Na tela de nova venda, no campo que filtra produtos pelo nome, foi digitado o texto abaixo:

```
' OR '1'='1
```

O resultado obtido foi uma lista de produtos vazia, com o aviso de que nenhum produto corresponde ao filtro informado. Nenhuma linha a mais foi devolvida, nenhum erro de sintaxe apareceu e nada no banco foi alterado.

A explicação está em como o driver conversa com o servidor. O texto do comando e os valores viajam em mensagens separadas, de modo que o conteúdo digitado nunca chega ao analisador de sintaxe do banco. O texto foi procurado como nome de produto: o banco buscou um produto literalmente chamado *' OR '1'='1*, não encontrou, e devolveu vazio.

Se o filtro fosse concatenado ao comando, a situação seria outra. O comando chegaria ao servidor com a condição abaixo:

```
WHERE produto LIKE '%' OR '1'='1%'
```

A comparação de 1 com 1 é sempre verdadeira, e o OR faria a cláusula aceitar qualquer linha. O catálogo inteiro voltaria, ignorando o filtro. Em uma consulta que filtrasse por cliente ou por pedido, o mesmo recurso devolveria dados de outras pessoas.

4 Transações e controle de concorrência

4.1 A operação de venda

Registrar uma venda é a operação central desta etapa. Ela toca cinco tabelas, e ou todas mudam, ou nenhuma muda.

A transação começa com o desligamento da confirmação automática e com a declaração explícita do nível de isolamento:

```
con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
con.setAutoCommit(false);
```

O primeiro comando grava o cabeçalho do pedido, com a data e hora do momento e a situação inicial de aberto. Em seguida, para cada produto da venda, vem o comando mais importante de todos:

```
SELECT e.idestoque, e.quantidade_atual, p.nome, p.preco_venda
  FROM estoque e
  JOIN produto p ON p.idproduto = e.produto_idproduto
 WHERE e.produto_idproduto = ?
   FOR UPDATE
```

A cláusula FOR UPDATE não apenas lê o saldo: ela tranca a linha do estoque até o fim da transação. Outra venda do mesmo produto que acontecer ao mesmo tempo fica esperando neste ponto. É este comando, e não o nível de isolamento, que impede que o mesmo saldo seja vendido duas vezes.

Com a linha trancada, a aplicação compara o saldo com a quantidade pedida. Se não cobrir, recusa aqui mesmo, informando quanto há disponível.

Cobrindo, seguem os quatro comandos restantes. O item do pedido é gravado com o preço lido durante a trava, e não com o preço de tabela consultado depois — a RN08 exige que valha o preço praticado no dia da venda. O saldo do estoque é abatido. A movimentação é registrada como saída, com data e hora, como pede a RN10. E a cobrança é criada com situação pendente, respeitando a RN13, que admite um pagamento por pedido.

No fim, a transação é confirmada:

```
con.commit();
```

Se qualquer um dos passos falhar, o desfazimento acontece no tratamento da exceção, e o encerramento devolve a conexão ao modo automático antes de fechá-la. Não existe caminho de saída do método que deixe uma transação aberta.

4.2 Quando a transação falha

Há duas maneiras diferentes de uma venda não se concretizar, e elas não são a mesma coisa.

A primeira é a recusa por estoque insuficiente. Ela acontece logo depois da leitura do saldo, antes de qualquer gravação, e por isso não há nada a desfazer. A segunda é o erro no meio do processo, depois que algumas linhas já foram escritas — e é essa que exige o desfazimento.

Para poder demonstrar a segunda situação, a tela tem uma caixa chamada *Provocar falha no fim da transação*. Com ela marcada, todos os passos são executados normalmente e o erro é lançado depois, justamente para que haja algo a desfazer.

A prova de que o banco permaneceu íntegro é a consulta de verificação V01, escrita na Etapa 1. Ela compara, para cada produto, o saldo registrado no estoque com a soma de todas as suas movimentações. Depois de qualquer uma das duas falhas, ela continua voltando vazia. Qualquer linha devolvida significaria que uma gravação parcial ficou para trás.

4.3 Duas vendas ao mesmo tempo

Para observar o comportamento do banco sob acesso simultâneo, foram abertas duas sessões independentes e, em cada uma, iniciada uma transação que tenta trancar a mesma linha de estoque:

```
START TRANSACTION;
SELECT quantidade_atual FROM estoque
 WHERE produto_idproduto = 1
   FOR UPDATE;
```

A segunda sessão fica esperando. Ela não recebe erro, não lê um valor desatualizado e não segue adiante: simplesmente aguarda a primeira terminar. Ao confirmar a transação da primeira sessão, a segunda destrava imediatamente e lê o saldo já abatido.

Se a primeira sessão for deixada aberta, a segunda desiste ao fim do tempo configurado no servidor e devolve a mensagem de espera excedida pelo bloqueio. Por padrão esse tempo é de cinquenta segundos.

[Registrar aqui o nível de isolamento em vigor obtido na consulta, o comportamento observado, o tempo de espera medido e, se houve, a mensagem de espera excedida. Anexar as imagens das duas sessões.]

4.4 Por que READ COMMITTED

A aplicação declara o nível READ COMMITTED em vez de aceitar o padrão do InnoDB, que é REPEATABLE READ. A escolha merece justificativa.

O nível padrão garante que uma mesma leitura repetida dentro da transação devolva sempre o mesmo valor. Aqui isso seria indesejável. A transação de venda lê o saldo do estoque e decide a partir dele, e duas vendas simultâneas do mesmo produto precisam enxergar o abatimento uma da outra. Segurar a leitura no valor do início da transação atrasaria essa percepção sem trazer benefício algum para este caso de uso.

O ponto que costuma ser mal compreendido é supor que o nível de isolamento seja o que impede a venda duplicada. Não é. Quem serializa as duas transações é o bloqueio de linha do FOR UPDATE, que funciona em qualquer nível. O isolamento mais baixo apenas evita um trabalho de versionamento que, neste desenho, não seria aproveitado.

E a última linha de defesa continua sendo o próprio banco: a restrição que impede quantidade negativa no estoque recusaria a operação mesmo que a aplicação errasse a conta, cumprindo a RN11.

5 Otimização de consultas

[Seção pendente. Responsável: a definir.]

Esta seção deve apresentar o estudo de desempenho de três consultas críticas. Para cada uma: o plano de execução antes da intervenção, acompanhado da leitura do plano em texto; o índice criado, com justificativa da escolha das colunas e da ordem delas; o plano depois, com comparação do tempo medido e o volume de dados informado; e uma discussão honesta do resultado.

As três candidatas naturais já estão escritas e funcionando no arquivo de consultas da Etapa 1. A primeira cruza a cobertura de estoque com o prazo dos fornecedores, e é a pergunta de negócio não trivial do projeto. A segunda percorre a hierarquia de supervisão entre funcionários, usando consulta recursiva. A terceira monta a curva ABC de produtos com funções de janela.

Cabe registrar uma expectativa. Com pouco mais de mil registros, é provável que algum índice não traga ganho nenhum, porque o otimizador escolhe varrer a tabela inteira quando ela cabe folgada na memória. O enunciado é explícito em que resultado negativo bem analisado vale tanto quanto ganho expressivo, de modo que esse achado é material para a discussão, e não um problema a esconder.

6 Segurança e controle de acesso

Esta seção está parcialmente concluída. O armazenamento de senha está implementado e descrito abaixo; os perfis de banco, a visão de restrição e o registro de dados pessoais seguem pendentes.

6.1 Senhas com hash e sal

A senha em texto claro nunca é gravada e nunca sai da classe que cuida dela. O que vai para a tabela de usuários são dois valores: um sal de dezesseis bytes sorteado para aquele usuário, e o resultado da função PBKDF2 aplicada sobre a senha somada ao sal, com cento e vinte mil repetições.

Três decisões foram tomadas aqui, e cada uma tem um motivo.

O sal é sorteado por usuário, e não fixo no sistema. Com isso, duas pessoas que escolham a mesma senha passam a ter resultados diferentes gravados. Isso inutiliza tabelas de consulta pré-calculadas e impede que alguém descubra, só de olhar o banco, que dois usuários usam a mesma senha.

A função escolhida é propositalmente lenta. A lentidão é o recurso: ela encarece a tentativa de descobrir a senha por força bruta. Uma função rápida cumpriria a letra do enunciado e seria uma escolha ruim.

A comparação entre a senha digitada e a gravada percorre os dois textos inteiros, mesmo depois de encontrar uma diferença, para que o tempo de resposta não revele quantos caracteres estavam certos.

Pela mesma razão, a recusa de login é deliberadamente genérica. A tela diz que o login ou a senha estão incorretos, sem distinguir os dois casos, porque distingui-los entregaria quais logins existem.

6.2 Perfis de acesso

[Pendente. Responsável: a definir.]

São necessários pelo menos três perfis distintos, criados com GRANT e REVOKE, aplicando o princípio do menor privilégio. A aplicação já tem os perfis nomeados e o comportamento de tela correspondente, o que dá o desenho de partida: administrador, vendedor e estoquista. As abas visíveis mudam conforme o perfil, de modo que o estoquista não vê clientes e apenas o administrador vê funcionários.

É importante dizer isto com todas as letras no relatório: esconder uma aba não é controle de acesso. O controle que vale é o do banco. A tela apenas evita que a pessoa tente uma operação que o banco recusaria de qualquer modo. As concessões precisam existir para que a afirmação seja verdadeira.

6.3 Visão de restrição

[Pendente. Responsável: a definir.]

É preciso ao menos uma visão que restrinja colunas ou linhas visíveis a determinado perfil. O caminho mais curto e mais defensável pelo domínio é uma visão de cliente sem CPF, telefone e endereço, liberada ao

estoquista, que precisa saber para quem é a entrega mas não precisa dos dados pessoais.

6.4 Dados pessoais

[Pendente. Responsável: a definir.]

O enunciado pede o registro de quais dados do projeto seriam considerados pessoais e que cuidados foram adotados. O levantamento é curto, porque o modelo é enxuto: a tabela de pessoa guarda nome, CPF, telefone e e-mail; a de cliente guarda endereço; a de funcionário guarda cargo e data de admissão; e a de usuário guarda o login. O CPF é o mais sensível de todos, porque identifica univocamente e não muda ao longo da vida.

Os cuidados já adotados que cabem nesse registro são a senha nunca armazenada em texto claro, as credenciais mantidas fora do controle de versão e a visão de restrição descrita acima, quando existir.

7 Gatilhos e auditoria

[Seção pendente. Responsável: a definir.]

São necessários dois gatilhos que registrem alterações relevantes em uma tabela de auditoria, guardando o autor, a data e os valores anterior e posterior. O enunciado cobra que a escolha do que auditar esteja justificada pelo domínio, e é a justificativa que pesa mais do que o gatilho em si.

Duas escolhas se defendem sozinhas. A primeira é a alteração de preço de produto. Como a RN08 manda o item do pedido guardar o preço praticado no dia da venda, o preço atual do produto muda ao longo do tempo sem deixar rastro em lugar nenhum: olhando o banco hoje, não é possível dizer quanto o produto custava no mês passado, nem quem o alterou. Auditar esse campo fecha uma lacuna real.

A segunda é a mudança de saldo de estoque, que é onde um erro fica invisível. As movimentações registram entradas e saídas, mas uma alteração direta no saldo, feita fora da aplicação, mudaria o número sem movimentação correspondente — exatamente o que a consulta de verificação V01 procura. Um gatilho transforma essa divergência de algo que só aparece quando alguém roda a verificação em algo registrado no momento em que acontece.

Dois detalhes práticos merecem atenção na hora de escrever. O autor obtido dentro do gatilho é o usuário do MySQL, e não o usuário da aplicação; vale dizer no relatório qual dos dois está sendo registrado e por quê. E a coluna de data da auditoria deve guardar data e hora, pelo mesmo motivo que levou a RN10 a exigir isso das movimentações.

8 Projeto físico e armazenamento

[Seção pendente. Responsável: a definir.]

Esta seção reúne quatro assuntos: a revisão dos tipos de dados à luz do que se aprendeu no semestre, a estimativa de crescimento das tabelas de maior movimento em dois anos, o plano de backup e, se alguma tabela justificar, a proposta de particionamento ou arquivamento.

Na revisão dos tipos, dois pontos valem ser examinados. O primeiro é a distinção entre guardar apenas a data e guardar data e hora: a RN10 exige o momento das movimentações, e vale percorrer todas as colunas de data do esquema verificando se cada uma guarda o que a regra correspondente exige. O segundo é o tipo usado nos indicadores de situação, que guardam zero ou um com restrição de verificação; cabe discutir se um tipo booleano seria mais expressivo, lembrando que no MySQL ele é apenas um apelido, o que torna a discussão sobre legibilidade e não sobre armazenamento.

A estimativa de crescimento deve partir das proporções observadas na carga, em vez de números arbitrários. A carga tem mil duzentos e setenta e um registros, dos quais trezentos e setenta e quatro são movimentações de estoque, cento e noventa e nove são itens de pedido e setenta são pedidos — ou seja, cerca de dois itens e oito décimos por pedido. A tabela de movimentações é a que cresce mais rápido, e por um motivo estrutural: cada venda gera uma saída por produto vendido e cada reposição gera uma entrada. Ela cresce com o movimento, não com o cadastro, enquanto produto e cliente crescem devagar.

O plano de backup precisa dizer o que copiar, com que frequência, com que estratégia de retenção e, sobretudo, como a restauração seria testada. Esse último item é o que costuma faltar, e é o único que prova que o plano funciona. A própria entrega oferece um argumento pronto: o script completo foi executado em base limpa e conferido pela tabela de contagens que ele imprime no fim, o que é exatamente a forma de um teste de restauração.

Quanto ao particionamento, a candidata óbvia é a tabela de movimentações, separada por período, já que as consultas de operação olham os últimos meses e o histórico antigo só é lido em relatório. Se a conclusão for que o volume ainda não justifica, essa também é uma resposta válida, desde que acompanhada do número a partir do qual passaria a justificar.

9 Revisão da Etapa 1

A devolutiva da Etapa 1 reconheceu a entrega como completa e apontou pontos de melhoria em cinco critérios. Esta seção percorre cada observação e registra o que foi feito em resposta.

9.1 Modelagem conceitual

Observação: o MER do dbdiagram.io está com cara de modelo físico, sem cardinalidades mínima e máxima legíveis; refazer em notação única, a mesma do modelo lógico, com cardinalidades em todos os relacionamentos.

A observação estava correta, e a causa era simples: o arquivo entregue como modelo conceitual era a mesma figura do modelo lógico. Trazia tipos de dados, chaves estrangeiras e colunas de ligação, que são decisões de implementação e não pertencem ao nível conceitual.

O MER foi refeito do zero, em notação de Chen, usada do começo ao fim sem mistura com nenhuma outra. As entidades são retângulos, os relacionamentos são losangos com nome próprio, e todos os atributos aparecem como elipses ligadas à entidade ou ao relacionamento a que pertencem. Os identificadores vêm sublinhados, e o identificador parcial da entidade fraca vem em elipse tracejada. O diagrama traz uma legenda com todas essas convenções.

Cada relacionamento traz a cardinalidade mínima e máxima nos dois extremos, no formato (mín,máx), de modo que se lê diretamente quantas ocorrências de cada entidade participam dele.

O conceitual cobre as mesmas treze tabelas do modelo lógico e os seus cinquenta e oito atributos, sem deixar nenhum de fora — mas organizados como conceitos, e não como tabelas. As catorze chaves estrangeiras não aparecem como atributos: elas **são** as linhas dos relacionamentos. E as duas tabelas associativas do modelo lógico, *produto_fornecedor* e *item_pedido*, voltam a ser no conceitual o que de fato são, relacionamentos N:N com atributos próprios, desenhados como losangos verdes com as suas elipses.

Não há tipos de dados em lugar nenhum do diagrama. Isso é assunto do modelo lógico, e era justamente o que fazia o diagrama anterior parecer um modelo físico.

Observação: declarar no MER a especialização PESSOA, informando se é total ou parcial e exclusiva ou sobreposta.

A especialização passou a ser declarada explicitamente no diagrama, com o triângulo ligando PESSOA a CLIENTE e FUNCIONÁRIO, marcado como **total e exclusiva**.

É total porque a RN17 determina que toda pessoa cadastrada seja classificada segundo o seu tipo: não existe pessoa que não seja cliente nem funcionário. E é exclusiva porque a mesma regra trata os dois tipos como alternativos, de modo que ninguém é as duas coisas ao mesmo tempo.

Cabe um registro honesto aqui. O banco implementa a especialização com duas tabelas derivadas que compartilham a chave de PESSOA, e essa estrutura garante que cliente e funcionário existam apenas como especializações. Ela **não** garante, porém, nem a totalidade nem a exclusividade: nada no esquema impede que uma pessoa seja cadastrada sem aparecer em nenhuma das duas, ou que apareça nas duas. Fazer o banco cumprir isso exigiria um atributo discriminador em PESSOA com restrição de verificação, ou gatilhos. A regra está declarada no modelo e hoje é mantida pela aplicação.

Observação: revisar a entidade fraca; ITEM_PEDIDO é uma associativa.

A observação procede. ITEM_PEDIDO nasceu do relacionamento N:N entre PEDIDO e PRODUTO e é identificado pela combinação das duas chaves, como diz a RN18. Isso faz dele uma **entidade associativa**, e não uma entidade fraca: ele não depende de um pai para ser identificado, depende de dois, e por igual. No MER refeito ele e PRODUTO_FORNECEDOR aparecem rotulados como associativas.

A entidade fraca do modelo é a **MOVIMENTACAO_ESTOQUE**. Uma movimentação não tem existência nem sentido fora da posição de estoque a que pertence, e o que a distingue das demais movimentações daquele mesmo estoque é o instante em que ocorreu. O identificador dela é, portanto, o identificador do ESTOQUE somado a *data_hora*, que funciona como identificador parcial. Não por acaso a RN10 exige data **e** hora: é justamente o que torna a movimentação distinguível.

Na implementação física ela recebeu uma chave substituta, como todas as demais tabelas, por uniformidade e porque a chave natural seria composta e longa para ser referenciada. A dependência de identificação, no entanto, é real e está declarada no modelo conceitual, onde a entidade aparece com borda dupla.

9.2 Modelo lógico e normalização

Observação: demonstrar a 3FN relação por relação, com as dependências funcionais, em vez de só afirmá-la.

A demonstração segue abaixo. A notação é a usual: à esquerda da seta ficam os atributos determinantes, à direita os determinados, e as chaves primárias aparecem sublinhadas no modelo lógico.

PESSOA (*idpessoa*, nome, cpf, telefone, email)

```
idpessoa → nome, cpf, telefone, email
cpf      → idpessoa
email    → idpessoa
```

Todos os atributos são atômicos, o que satisfaz a 1FN. A chave primária é simples, de modo que não existe dependência parcial e a 2FN é imediata. Nenhum atributo não-chave determina outro: saber o nome não determina o telefone, nem o contrário. Logo não há dependência transitiva, e a relação está na 3FN. Como cpf e email são únicos, os três determinantes são chaves candidatas, e a relação alcança também a **FNBC**.

CLIENTE (*pessoa_idpessoa*, endereco, data_cadastro)

```
pessoa_idpessoa → endereco, data_cadastro
```

Chave simples, herdada de PESSOA pela especialização. Nenhuma dependência parcial nem transitiva. Está na 3FN e na **FNBC**, porque o único determinante é a chave.

FUNCIONARIO (*pessoa_idpessoa*, cargo, data_admissao, ativo, idsupervisor)

```
pessoa_idpessoa → cargo, data_admissao, ativo, idsupervisor
```

Mesmo caso. Vale observar que *cargo* não determina nenhum outro atributo: se o modelo guardasse, por exemplo, o salário do cargo, haveria a dependência transitiva cargo → salário, e seria preciso extrair uma relação CARGO. Como o cargo é apenas um rótulo, isso não ocorre. Está na 3FN e na **FNBC**.

CATEGORIA (*idcategoria*, nome, descricao)

```
idcategoria → nome, descricao
nome        → idcategoria
```

O nome é único, então é chave candidata. Os dois determinantes são chaves candidatas, e a relação está na 3FN e na **FNBC**.

PRODUTO (*idproduto*, nome, preco_venda, unidade_medida, categoria_idcategoria)

```
idproduto → nome, preco_venda, unidade_medida, categoria_idcategoria
```

Aqui o nome **não** é único, de propósito: dois fornecedores podem trazer produtos de mesmo nome e apresentações diferentes. Há portanto uma única chave candidata. Nenhum atributo não-chave determina outro — em particular, a categoria não determina o preço nem a unidade de medida. Está na 3FN e na **FNBC**.

FORNECEDOR (*idfornecedor*, razao_social, cnpj, telefone, email)

```
idfornecedor → razao_social, cnpj, telefone, email
cnpj         → idfornecedor
email        → idfornecedor
```

Mesma situação de PESSOA: três chaves candidatas, todas determinantes. 3FN e **FNBC**.

PRODUTO_FORNECEDOR (*produto_idproduto*, *fornecedor_idfornecedor*, preco_compra, prazo_fornecimento)

```
{produto_idproduto, fornecedor_idfornecedor} → preco_compra, prazo_fornecimento
```

Esta é a primeira relação de chave composta, e por isso a primeira em que a 2FN exige atenção. A pergunta é se algum atributo não-chave depende de apenas parte da chave.

Não depende, e a RN06 explica por quê: o preço de compra e o prazo de fornecimento são próprios **da relação** entre aquele produto e aquele fornecedor. O mesmo produto tem preços diferentes conforme quem o fornece, e o mesmo fornecedor pratica prazos diferentes conforme o produto. Nenhuma das duas metades da chave determina sozinha qualquer um dos dois atributos, de modo que a dependência é total e a 2FN está satisfeita. Não há dependência transitiva. 3FN e **FNBC**.

PEDIDO (*idpedido*, data_pedido, status, cliente_pessoa_idpessoa)

```
idpedido → data_pedido, status, cliente_pessoa_idpessoa
```

Chave simples. Note-se que o pedido **não** guarda o nome nem o endereço do cliente: se guardasse, haveria a dependência transitiva idpedido → cliente → nome, violando a 3FN. Esses dados ficam em PESSOA e CLIENTE e são alcançados por junção. 3FN e **FNBC**.

ITEM_PEDIDO (*pedido_idpedido*, *produto_idproduto*, quantidade, preco_unitario)

```
{pedido_idpedido, produto_idproduto} → quantidade, preco_unitario
```

Esta é a relação mais interessante do esquema do ponto de vista da normalização, e merece o argumento completo.

À primeira vista, *preco_unitario* pareceria depender apenas de *produto_idproduto* — afinal, o preço é do produto. Se fosse assim, haveria uma dependência parcial, a relação violaria a 2FN, e a correção seria simplesmente apagar a coluna, já que o preço de tabela está em PRODUTO.preco_venda.

Mas não é assim. A RN08 determina que o item registre o preço praticado **no momento da compra**. Esse valor não é o preço atual do produto: é o preço daquele produto naquele pedido, em uma data específica. Depende, portanto, da chave inteira. A relação está na 2FN.

Essa distinção não é formalidade. Sem a coluna, alterar o preço de um produto mudaria retroativamente o valor de todas as vendas passadas, e o faturamento histórico deixaria de ser reprodutível. Não há dependência transitiva, e a relação está na 3FN e na **FNBC**.

ESTOQUE (*idestoque*, produto_idproduto, quantidade_atual, data_atualizacao)

```
idestoque         → produto_idproduto, quantidade_atual, data_atualizacao
produto_idproduto → idestoque
```

Como *produto_idproduto* é único, ele é chave candidata, e a relação tem dois determinantes, ambos chaves. 3FN e **FNBC**. Esta relação é retomada adiante, no item sobre desnormalização.

MOVIMENTACAO_ESTOQUE (*idmovimentacao_estoque*, tipo, quantidade, data_hora, estoque_idestoque)

```
idmovimentacao_estoque → tipo, quantidade, data_hora, estoque_idestoque
```

Chave substituta simples. Conforme discutido no item sobre a entidade fraca, a chave natural seria {estoque_idestoque, data_hora}; ela não foi declarada como única no banco, de modo que formalmente não é chave candidata. 3FN e **FNBC**.

PAGAMENTO (*idpagamento*, forma_pagamento, status_pagamento, data_pagamento, pedido_idpedido)

```
idpagamento    → forma_pagamento, status_pagamento, data_pagamento, pedido_idpedido
pedido_idpedido → idpagamento
```

A RN13 admite um pagamento por pedido, o que torna *pedido_idpedido* único e, portanto, chave candidata. Os dois determinantes são chaves. 3FN e **FNBC**.

ENTREGA (*identrega*, data_prevista, data_entrega, status_entrega, pedido_idpedido, funcionario_pessoa_idpessoa)

```
identrega       → data_prevista, data_entrega, status_entrega,
                  pedido_idpedido, funcionario_pessoa_idpessoa
pedido_idpedido → identrega
```

Mesma estrutura de PAGAMENTO, pela RN14. 3FN e **FNBC**.

9.3 Desnormalização deliberada

O enunciado pede que se aponte, se houver, alguma relação deliberadamente desnormalizada, com a justificativa de desempenho. Há uma, e é consciente.

O atributo *quantidade_atual*, em ESTOQUE, é um valor derivado: ele pode ser calculado somando todas as entradas e subtraindo todas as saídas registradas em MOVIMENTACAO_ESTOQUE. Guardá-lo é, a rigor, redundância controlada.

A justificativa é de desempenho e de concorrência. O saldo é consultado a toda hora — em cada venda, em cada tela de produto, no filtro da tela de venda — enquanto o histórico cresce indefinidamente. Calcular o saldo por agregação a cada leitura significaria percorrer todas as movimentações do produto, e esse custo cresce com o tempo de uso do sistema. Além disso, a trava de linha que serializa duas vendas simultâneas, descrita na seção 4, incide sobre a linha de ESTOQUE: sem uma linha para travar, seria preciso bloquear um conjunto de movimentações, o que é mais caro e mais sujeito a impasse.

O risco da redundância é a divergência entre o saldo e o histórico. Esse risco é tratado de duas formas: a atualização do saldo e o registro da movimentação acontecem sempre dentro da mesma transação, e existe uma consulta de verificação que compara os dois a qualquer momento.

9.4 Correções de nomenclatura

Observação: corrigir inconsistências de nomes (preco_compra x preco_comprar, pedida_idpedido).

A primeira era real. A coluna da tabela PRODUTO_FORNECEDOR estava declarada no script como *preco_comprar*, que é forma verbal e destoa de *preco_venda*, enquanto o modelo lógico e o dicionário de dados sempre a chamaram de *preco_compra*. O nome correto é **preco_compra**, e foi ele que passou a valer: a alteração atingiu o script de estrutura e as onze inserções do script de carga, e o banco foi reconstruído e conferido depois dela.

A segunda não foi localizada. Nos arquivos entregues, a coluna aparece sempre como *pedido_idpedido*, tanto no script quanto no modelo lógico e no dicionário. A forma *pedida_idpedido* provavelmente constava de uma versão anterior do material, corrigida antes da entrega final.

9.5 Comentários no script de estrutura

Observação: acrescentar comentários no 01_ddl.sql ligando cada restrição à RN que ela atende.

Feito para as quarenta e sete restrições nomeadas e para os oito índices. Cada uma recebeu, na linha acima, a regra correspondente e o que ela garante. Por exemplo:

```
-- RN11: o saldo nunca fica negativo: ultima linha de defesa da venda
CONSTRAINT ck_estoque_quantidade
    CHECK (quantidade_atual >= 0),

-- RN18: chave composta {pedido, produto}: impede o mesmo produto repetido no pedido
CONSTRAINT pk_item_pedido
    PRIMARY KEY (pedido_idpedido, produto_idproduto),
```

O ganho não é apenas documental. Quando uma restrição é violada, o MySQL devolve o nome dela na mensagem de erro, e é por esse nome que a aplicação identifica qual regra foi quebrada, como descrito na seção 2.4. O comentário fecha o caminho de volta: do erro para a restrição, da restrição para a regra de negócio.

9.6 Consulta de verificação

Observação: incluir, se possível, uma consulta de verificação que compare as movimentações com a quantidade_atual de cada produto.

Essa consulta já existia na entrega, como a V01 do arquivo de consultas, e é exatamente o controle da redundância discutida no item 9.3. Ela soma as entradas, subtrai as saídas e compara o resultado com o saldo registrado, devolvendo apenas os produtos em que os dois valores divergem — de modo que o resultado esperado é nenhuma linha.

Ela passou a ter um segundo uso nesta etapa: é a prova de integridade depois das demonstrações de falha de transação, descritas na seção 4.2. Como a observação sugere que ela pode não ter sido notada, o relatório final passa a destacá-la em vez de deixá-la entre as demais.

9.7 Documentação

Observação: seguir a numeração A1 a A5 do enunciado, usar o dicionário no formato do Anexo A, deixar a tabela de regras de negócio mais legível e informar o uso de IA também no README.

As seções da documentação da Etapa 1 foram renumeradas conforme o enunciado, de A1 a A5. O dicionário de dados foi reformatado no padrão do Anexo A. A tabela de regras de negócio foi refeita para caber na largura da página sem quebrar palavras no meio, que era o que prejudicava a leitura. E a declaração de uso de inteligência artificial, que constava apenas do relatório, passou a constar também do README do repositório.

10 A apresentação

A apresentação da Etapa 2 é uma demonstração ao vivo de quinze minutos, e não uma exposição de slides sobre o que foi feito. O ambiente precisa estar pronto antes: banco povoado, aplicação rodando e duas sessões abertas para o experimento de concorrência.

O primeiro momento do roteiro é o contexto, com dois minutos para o domínio, as principais entidades e o que mudou no modelo desde a Etapa 1. As mudanças estão reunidas nesta seção 9, que serve de roteiro para essa parte.

10.1 Como subir a aplicação em uma máquina limpa

São dois passos, e não é preciso instalar nada além do MySQL 8.0 e de um Java.

O primeiro é importar o script único. Ele apaga o banco, refaz a estrutura, carrega os dados e cria o usuário administrador. Ao terminar, imprime uma tabela de conferência que deve mostrar sessenta produtos, setenta pedidos, trezentas e setenta e quatro movimentações e um usuário. O segundo é abrir o projeto na IDE e executá-lo.

Na primeira execução a aplicação pede o acesso ao MySQL em uma janela, testa a conexão e grava o arquivo de configuração. Essa sequência é também a prova da condição eliminatória da Etapa 1, e vale repeti-la na véspera da apresentação.

11 Declaração de uso de inteligência artificial

A equipe utilizou ferramentas de inteligência artificial generativa como apoio ao longo do desenvolvimento deste trabalho.

O uso se deu em caráter auxiliar, em atividades de pesquisa, revisão, estruturação de texto e apoio à escrita de código. Todo o conteúdo produzido foi revisado, testado e validado pelos integrantes, que respondem integralmente pelas decisões de modelagem, pelas regras de negócio adotadas e pelo funcionamento do que foi entregue.

As ferramentas foram empregadas como instrumento de produtividade e aprendizado, sem substituir a compreensão do conteúdo da disciplina, que permanece sendo de responsabilidade da equipe.